

Ingeniería en Tecnologías de la Información y Comunicación

Nivel:

Tercero

Asignatura:

Metodología de Desarrollo de Software

NRC:

29531

Actividad:

Proyecto Tercera Unidad ElectaVox

Estudiantes:

Iker Guallo

Jeffry Vera

Docente:

Ing. Kevin Chuquitarco

Fecha:

29/07/2026

Santo Domingo - Ecuador

Contenido

Introducción 3
Objetivos 4
Objetivo General 4
Objetivos Específicos 4
Desarrollo 4
Preparación y Planificación del Entorno de Desarrollo ElectaVox 5
Configuración Técnica de las Herramientas de Desarrollo 9
Implementación técnica de todas las historias de usuario 14
Historia de Usuario 1: Registro de estudiantes 15
Historia de Usuario 2: Inicio de sesión 17
Historia de Usuario 3: Registro de candidatos 19
Historia de Usuario 4: Consulta de candidatos 21
Historia de Usuario 5: Emitir voto 23
Historia de Usuario 6: Validación de voto único 25
Historia de Usuario 7: Temporizador electoral 27
Historia de Usuario 8: Resultados en tiempo real 29
Historia de Usuario 9: Estadísticas por curso 31
Historia de Usuario 10: Estadísticas por género 33
Historia de Usuario 11: Generar reporte final 35
Historia de Usuario 12: Cierre de elección 38
Módulo de Seguridad y Acceso (Login) 40
Módulo de Interfaz Principal y Contenedor de Navegación 41
Persistencia de Datos y Estructura de MongoDB 42
Pruebas Funcionales Realizadas 43
Matriz de Implementación y Resultados 44
Conclusiones 45
Recomendaciones 46
Referencias 47
Enlaces del Proyecto 48

Introducción

Los procesos electorales estudiantiles requieren mecanismos confiables para registrar a los votantes, presentar candidaturas, controlar la emisión de votos y consolidar resultados. Cuando estas actividades se ejecutan de forma manual, el equipo organizador debe invertir tiempo en verificar identidades, impedir duplicidades, realizar el conteo y preparar el acta final; además, la trazabilidad depende de documentos dispersos y de controles susceptibles a errores humanos.

En respuesta a esta necesidad se desarrolló ElectaVox, una aplicación de escritorio que digitaliza el ciclo electoral estudiantil. El sistema permite administrar el padrón, autenticar usuarios, registrar y consultar candidatos, emitir un voto único, controlar el período mediante un temporizador, visualizar resultados y estadísticas, y producir un reporte final. La solución fue construida con Java Swing y Apache Maven en NetBeans, mientras que MongoDB se utiliza para la persistencia documental de usuarios, estudiantes, candidatos, elecciones y votos.

La organización del trabajo se realizó mediante Scrum, marco que estructura el desarrollo en iteraciones breves y orientadas a producir incrementos utilizables (Schwaber & Sutherland, 2020). En Jira Software se registraron cinco épicas, doce historias de usuario, criterios de aceptación, responsables, story points y dos sprints. El presente informe conserva la estructura académica del proyecto SGI-MA y documenta de manera secuencial la planificación, la configuración técnica, la implementación de cada historia, las evidencias del sistema, las pruebas ejecutadas y los resultados alcanzados.

Objetivos

Objetivo General

Implementar el sistema de elecciones estudiantiles ElectaVox mediante Java Swing, MongoDB, Apache Maven y la planificación ágil en Jira Software, con el propósito de automatizar el registro de participantes, garantizar la emisión de un voto único y generar resultados, estadísticas y reportes verificables del proceso electoral.

Objetivos Específicos

- Configurar el entorno de desarrollo en Apache NetBeans, organizar el proyecto Maven y establecer la conexión con MongoDB para disponer de una base técnica estable.
- Desarrollar los módulos de autenticación, padrón, candidatos, votación, temporizador, resultados, estadísticas, reportes y cierre electoral conforme a los criterios de aceptación definidos.
- Aplicar validaciones en la interfaz, en los servicios y en la base de datos para prevenir registros inconsistentes, accesos no autorizados y votos duplicados.
- Comprobar el cumplimiento de las historias de usuario mediante evidencias de funcionamiento y casos de prueba asociados al Product Backlog.
- Documentar la planificación de los dos sprints y el uso de Jira como herramienta de seguimiento del trabajo del equipo.

Desarrollo

Este capítulo presenta la ejecución metodológica y técnica de ElectaVox. Primero se describe la preparación del entorno ágil y la organización del Product Backlog; posteriormente se detallan la arquitectura, las herramientas y la persistencia. Finalmente se expone la implementación de cada historia de usuario con sus criterios, subtareas, evidencia visual y resultado funcional.

Preparación y Planificación del Entorno de Desarrollo ElectaVox

Historias de Usuario y Criterios de Aceptación

Las historias de usuario expresan una necesidad desde la perspectiva de quien utilizará el sistema y vinculan una funcionalidad con el beneficio esperado. En ElectaVox se definieron cinco épicas y doce historias, todas registradas en Jira junto con criterios de aceptación que permiten comprobar cuándo una funcionalidad puede considerarse terminada. La distribución total fue de 91 story points, asignados entre dos sprints según dependencia técnica, prioridad y riesgo.

Tabla 1

Épicas del Product Backlog de ElectaVox

ID	Épica	Descripción
E-01	Gestión de Usuarios	Registro del padrón y autenticación de estudiantes.
E-02	Gestión de Candidatos	Registro, fotografía, propuestas y consulta de candidatos.
E-03	Proceso de Votación	Emisión segura, voto único y control del tiempo.
E-04	Resultados y Estadísticas	Resultados en tiempo real y participación por curso y género.
E-05	Reportes y Cierre Electoral	Reporte final, acta, ganador y bloqueo de nuevos votos.

Priorización del Product Backlog

La priorización consideró que el padrón y la autenticación son prerrequisitos para la votación; de manera similar, los candidatos deben existir antes de su consulta y selección. Los resultados y estadísticas dependen de votos almacenados, mientras que el reporte y el cierre requieren datos consolidados. Este orden minimiza bloqueos y favorece que cada incremento sea funcional.

Tabla 2

Priorización del Product Backlog del Sistema ElectaVox

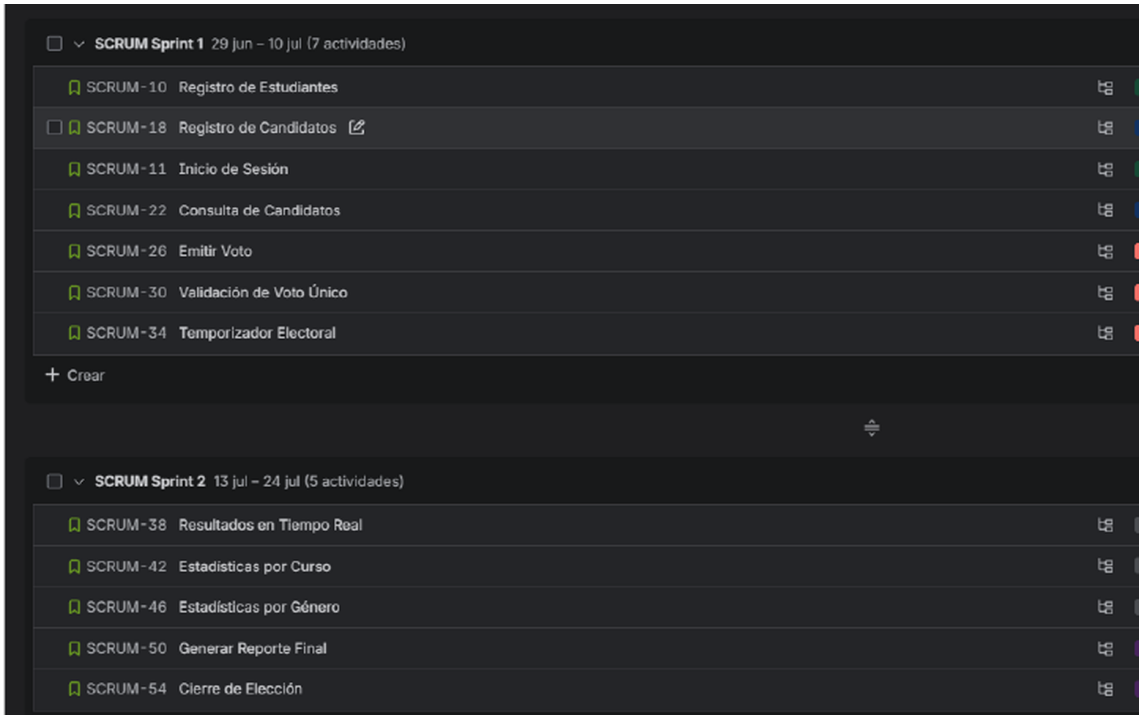
Orden	ID	Justificación
1	HU-01	Sustenta el padrón y habilita todas las funciones dependientes.

Orden	ID	Justificación
2	HU-02	Depende del registro y controla el acceso seguro.
3	HU-03	Debe existir antes de consultar o votar.
4	HU-04	Permite conocer las opciones disponibles.
5	HU-05	Integra autenticación, candidatos y elección activa.
6	HU-06	Garantiza la integridad mediante el bloqueo de duplicados.
7	HU-07	Controla la duración de la jornada electoral.
8	HU-08	Requiere votos para calcular conteos y porcentajes.
9	HU-09	Depende del padrón y de la participación registrada.
10	HU-10	Agrupa la participación utilizando datos del padrón.
11	HU-11	Consolida resultados y estadísticas al finalizar.
12	HU-12	Formaliza el cierre, ganador y bloqueo de nuevos votos.

La Figura 1 presenta la vista del backlog utilizada por el equipo para ordenar las historias por sprint, épica, estado, estimación y responsable.

Figura 1

Vista del Backlog Priorizado en Jira Software



Nota. Captura tomada del proyecto ElectaVox dentro del espacio de trabajo de Jira del equipo.

Planificación de los Sprints

El proyecto se distribuyó en dos sprints. El primero reunió las funciones necesarias para disponer de usuarios, candidatos y un flujo de votación controlado; el segundo incorporó la explotación de resultados, estadísticas y cierre. El descanso entre iteraciones permitió revisar el incremento del Sprint 1 y ajustar el Sprint Planning del segundo ciclo.

Como se puede visualizar en la Tabla 3, se muestran los dos Sprints del proyecto, con su fecha de inicio, fecha fin e historias que abarcan cada uno de ellos.

Tabla 3

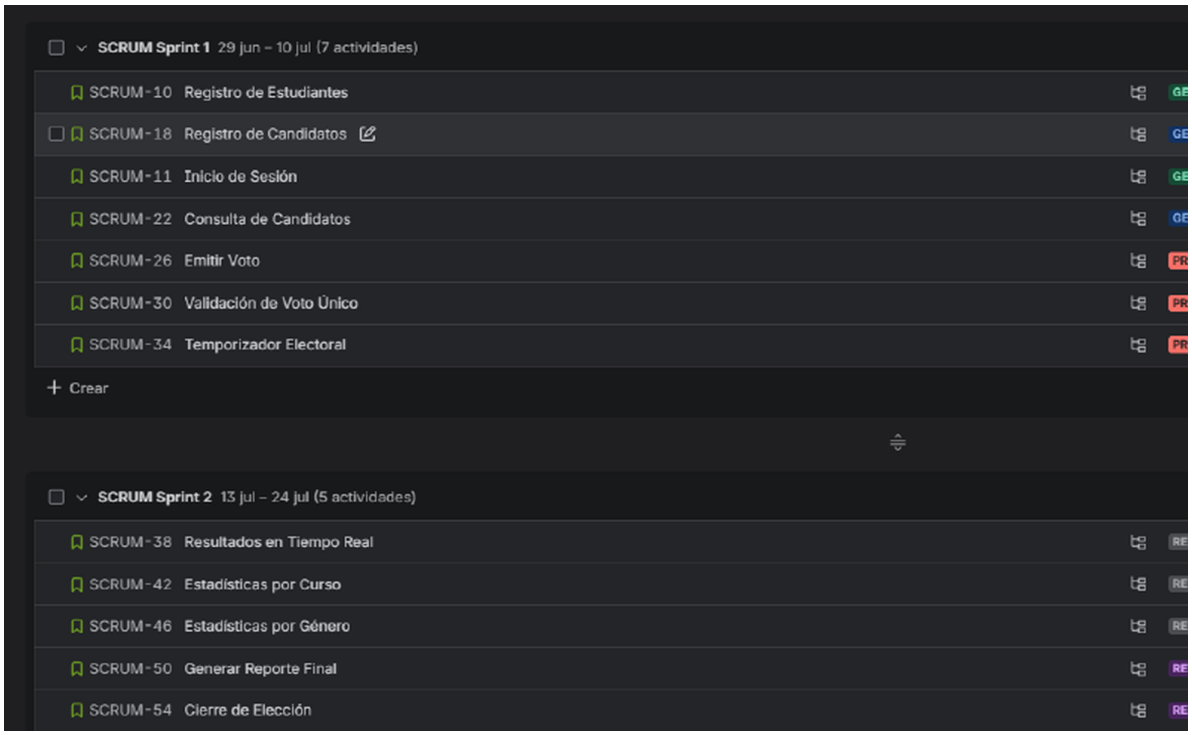
Planificación de los Sprints del Proyecto ElectaVox

Sprint	Fecha de inicio	Fecha de fin	Historias incluidas	Estimación
Sprint 1	29/06/2026	10/07/2026	HU-01 a HU-07	49 puntos
Sprint 2	13/07/2026	24/07/2026	HU-08 a HU-12	42 puntos

Como se observa en la Figura 2, las historias de usuario del proyecto ElectaVox fueron organizadas y distribuidas dentro de dos sprints, de acuerdo con su prioridad y con las funcionalidades planificadas para cada etapa del desarrollo.

Figura 2

Historias de usuario organizadas dentro de los dos Sprints



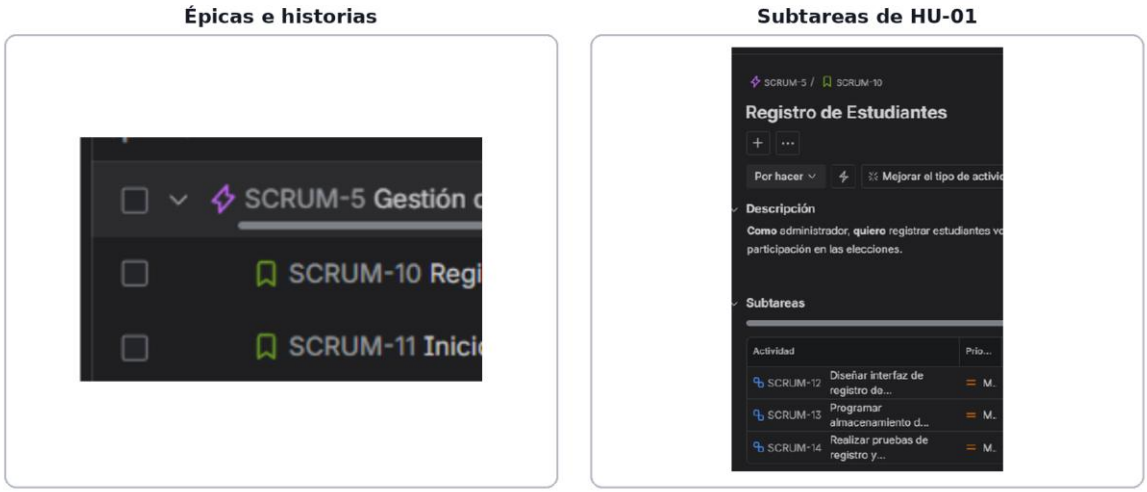
Registro de Épicas, Historias y Subtareas en Jira

Cada historia se vinculó con su épica, se asignó al sprint correspondiente y se descompuso en tres subtareas técnicas: diseño de la interfaz o módulo, programación de la funcionalidad y ejecución de pruebas. Esta estructura permite observar el avance desde una unidad de valor para el usuario hasta actividades concretas del equipo (Atlassian, 2020.).

En la Figura 3 en la izquierda se observa la agrupación por épica; en la derecha, se encuentran las subtareas de HU-01 registradas en Jira.

Figura 3

Historias de usuario y épicas relacionada en Jira.



Configuración Técnica de las Herramientas de Desarrollo

Después de consolidar la planificación se preparó el ecosistema técnico. ElectaVox utiliza un proyecto Maven con Java 21, formularios Swing editables en el GUI Builder de NetBeans y servicios especializados que aíslan la lógica de autenticación, estudiantes, candidatos, votación, resultados, elecciones y reportes. La persistencia se realiza en MongoDB mediante el controlador síncrono de Java.

Tabla 4

Tecnologías Empleadas en la Construcción de ElectaVox

Componente	Tecnología / Herramienta
Lenguaje de programación	Java 21 (JDK 21)
Interfaz gráfica	Java Swing y NetBeans GUI Builder
Gestión de dependencias	Apache Maven

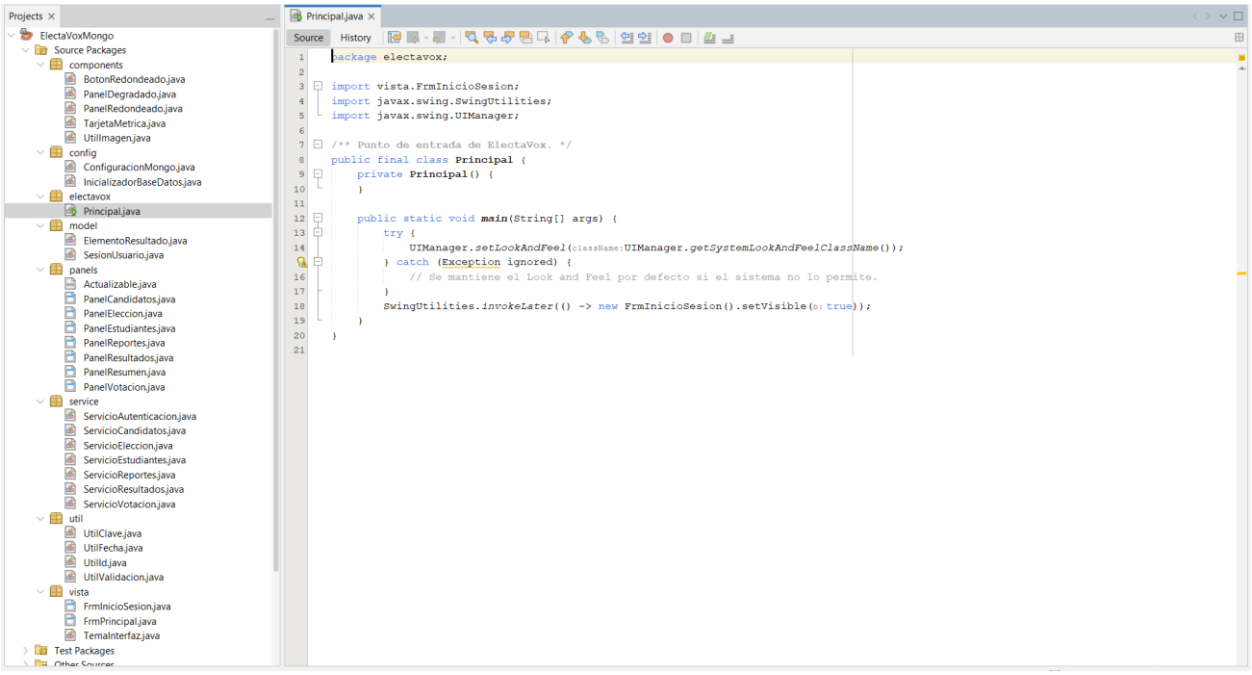
Componente	Tecnología / Herramienta
Base de datos	MongoDB: users, students, candidates, elections y votes
Arquitectura interna	Vistas y paneles, servicios, modelos, configuración, componentes y utilidades
Gestión ágil	Jira Software: épicas, historias, sprints y subtareas
Salida documental	Reportes y actas en archivos TXT

Entorno de Desarrollo Integrado: Apache NetBeans

NetBeans concentra la edición, compilación, ejecución y diseño visual. La inclusión de archivos. form junto con las clases Java permite abrir las ventanas y paneles en modo Design, mientras Maven mantiene las dependencias reproducibles. La organización observada en la Figura 3 separa configuración, servicios, modelos, paneles, vistas, componentes y utilidades, favoreciendo la mantenibilidad del código (Apache Software Foundation, 2022).

Figura 3

Estructura del Proyecto ElectaVox en Apache NetBeans



Arquitectura de la Aplicación

La aplicación sigue una separación por responsabilidades. FrmInicioSesion y FrmPrincipal administran la interacción general; los paneles implementan cada módulo funcional; los servicios encapsulan reglas y consultas; MongoDB mantiene los documentos persistentes. Las utilidades validan datos, procesan claves, fechas e identificadores, mientras los componentes personalizados unifican la apariencia.

Como se observa en la Figura 4, la arquitectura funcional del sistema ElectaVox se encuentra organizada en diferentes paquetes y clases, los cuales distribuyen las responsabilidades del sistema y facilitan su mantenimiento, comprensión y escalabilidad.

Figura 4

Arquitectura Funcional del Sistema ElectaVox

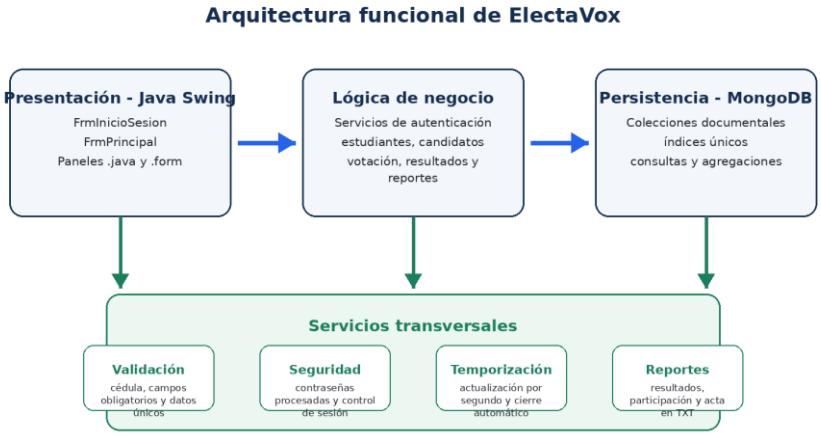
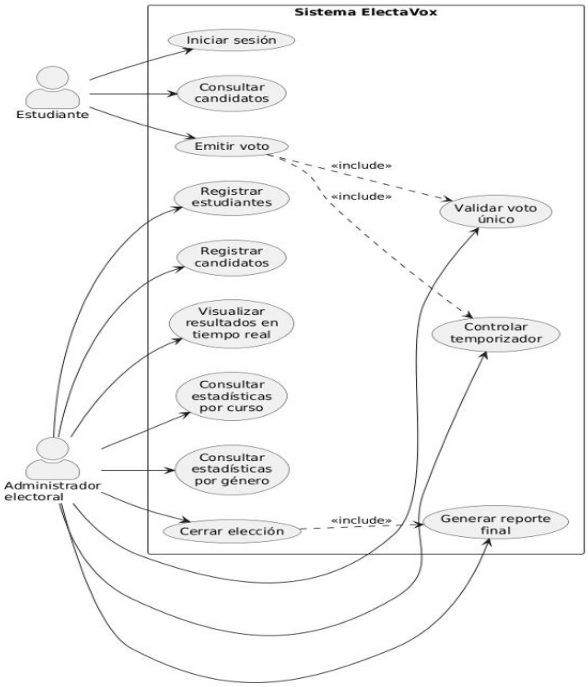


Diagrama de Casos de Uso

El diagrama de la Figura 5 distingue al administrador electoral y al estudiante. El administrador registra usuarios y candidatos, controla el tiempo, consulta resultados y cierra la elección; el estudiante se autentica, consulta candidatos y emite su voto. Los casos de generación de reportes, validación de voto único y control temporal complementan los flujos principales.

Figura 5

Diagrama de Casos de Uso del Sistema ElectaVox



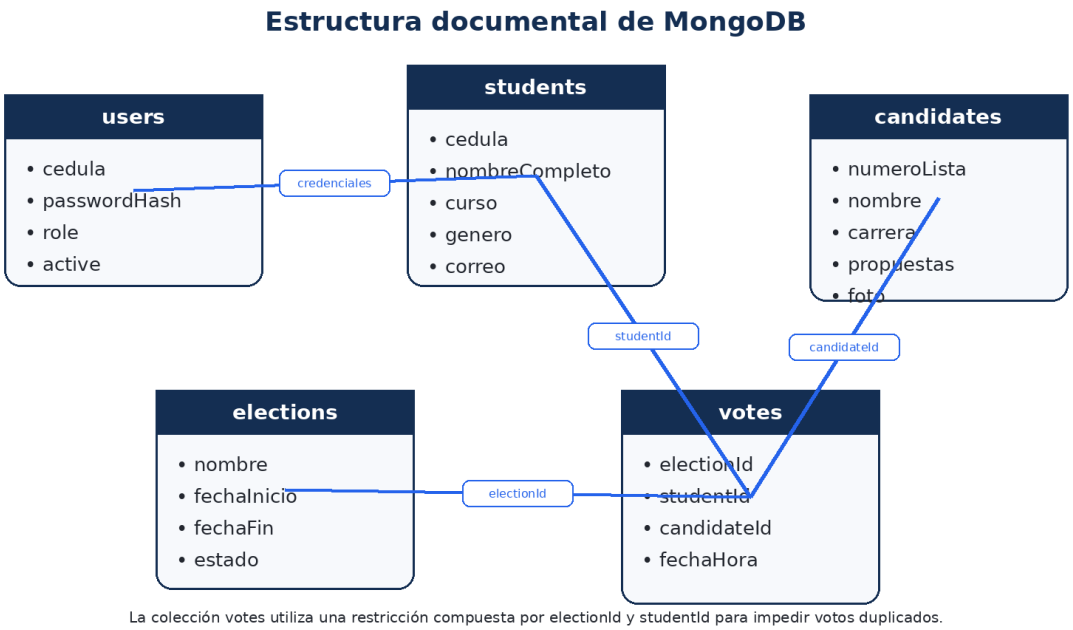
Sistema de Gestión de Base de Datos: MongoDB

MongoDB organiza la información mediante colecciones documentales, lo que permite almacenar atributos de candidatos, fotografías binarias, períodos electorales y registros de voto sin requerir una estructura tabular rígida. El controlador síncrono de Java proporciona operaciones de inserción, consulta, actualización, eliminación e índices desde los servicios del proyecto (MongoDB, Inc., 2024).

En la Figura 6 se observa el modelo documental de las colecciones de ElectaVox, en el cual se presentan las estructuras utilizadas para almacenar y organizar la información del sistema en MongoDB. Este modelo se basa en las clases ConfiguracionMongo e InicializadorBaseDatos, así como en los servicios responsables de gestionar las operaciones de persistencia.

Figura 6

Modelo Documental de las Colecciones de ElectaVox



La colección users conserva credenciales y roles; students representa el padrón habilitado; candidates contiene las listas y propuestas; elections define el período y el estado; votes vincula elección, estudiante y candidato. La combinación electionId-studentId se protege con un índice único para evitar duplicidades incluso si dos solicitudes se producen de forma cercana.

Implementación técnica de todas las historias de usuario

Las siguientes secciones presentan la correspondencia entre la planificación y el incremento construido. Cada historia incluye su ficha, criterios de aceptación, subtareas, descripción de la implementación y evidencia visual. Los datos visibles en las capturas son demostrativos y fueron cargados para verificar el comportamiento de la interfaz y de los cálculos.

Historia de Usuario 1: Registro de estudiantes

Estructuración y Despliegue en Jira Software

Como se observa en la Tabla 5, la historia de usuario HU-01 corresponde al registro de estudiantes. La historia se vinculó con la épica E-01 Gestión de Usuarios, se planificó para el Sprint 1 y se estimó en 5 story points. Además, se estableció una prioridad Highest y un nivel de riesgo bajo. Su implementación utiliza las colecciones students y users de MongoDB, junto con índices únicos para evitar la duplicación de la cédula y el correo electrónico de los estudiantes.

Tabla 5

Ficha de historia de usuario HU-01: Registro de estudiantes

ID	HU-01	**Usuario**	Administrador
Nombre historia	Registro de estudiantes	**Responsable**	Iker Guallo
Épica relacionada	E-01 Gestión de Usuarios	**Sprint asignado**	Sprint 1
Prioridad	Highest	**Riesgo**	Baja
Story points	5	**Estado**	Implementada
Descripción	Como administrador, quiero registrar estudiantes votantes, para habilitar su participación en las elecciones.		
Persistencia / módulo	Colecciones students y users; índices únicos de cédula y correo.		

Criterios de Aceptación

- Mostrar un formulario con cédula, nombre completo, curso y género.
- Mostrar confirmación al completar correctamente el registro.
- Alertar cuando existan campos obligatorios vacíos.
- Impedir el registro de una cédula ya existente.

Subtareas Técnicas

- Diseñar la interfaz de registro del estudiante.
- Programar las validaciones y el almacenamiento documental.
- Ejecutar pruebas de campos vacíos, cédula inválida y duplicidad.

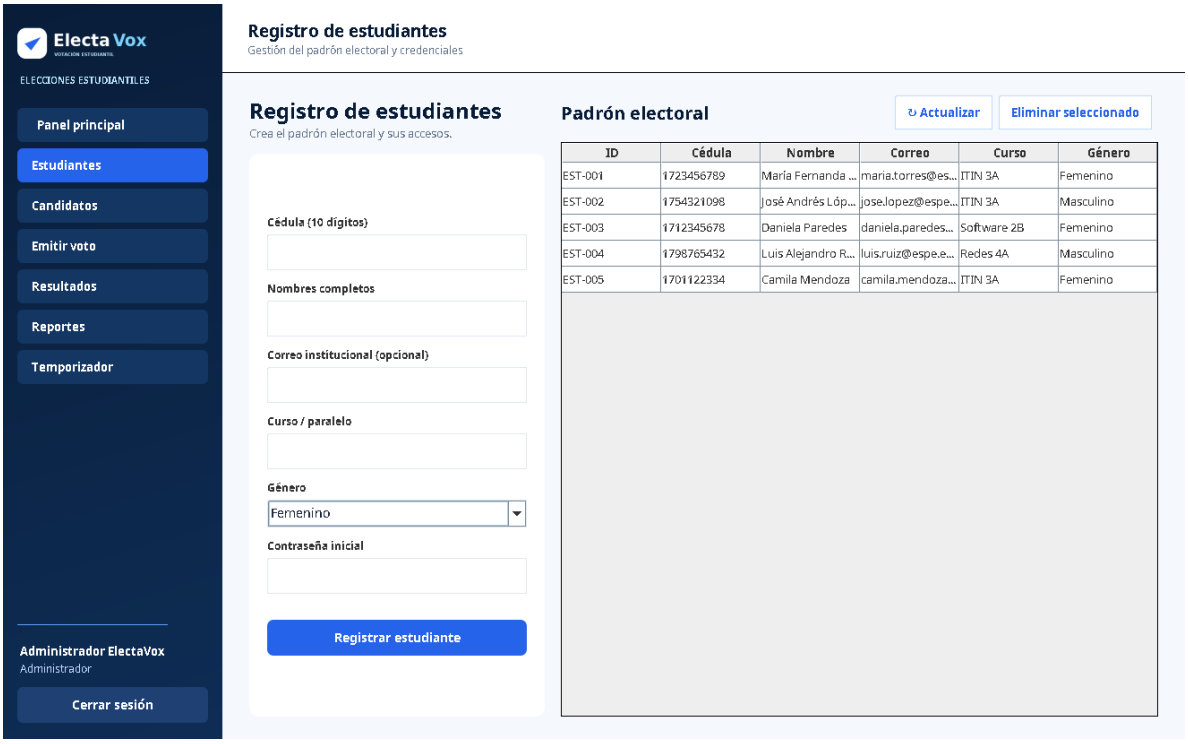
Desarrollo e Implementación de la Funcionalidad

El módulo PanelEstudiantes integra los campos definidos en la historia y amplía el criterio con contraseña obligatoria y correo institucional opcional. ServicioEstudiantes valida la cédula de diez dígitos, verifica duplicados y registra tanto el padrón electoral como las credenciales de acceso.

Como se observa en la Figura 7, la historia de usuario HU-01 fue implementada mediante el módulo **Registro de estudiantes**, que permite al administrador ingresar la cédula, los nombres completos, el correo institucional, el curso, el género y una contraseña inicial. La interfaz también presenta el padrón electoral con los estudiantes registrados y dispone de opciones para actualizar la información o eliminar un registro seleccionado, evidenciando el funcionamiento de la gestión de estudiantes en ElectaVox.

Figura 7

Evidencia de HU-01 - Registro de estudiantes



Validación del Resultado

La evidencia muestra el formulario operativo, la tabla de estudiantes y las acciones de registro, actualización y eliminación. Las validaciones preservan la consistencia del padrón antes de habilitar el acceso electoral.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Colecciones students y users; índices únicos de cédula y correo. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-01 y reproducir las pruebas desde NetBeans.

Historia de Usuario 2: Inicio de sesión

Estructuración y Despliegue en Jira Software

La historia HU-02 fue vinculada con la épica E-01, asignada a Sprint 1 y estimada en 5 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 6

Ficha de historia de usuario HU-02: Inicio de sesión

ID	HU-02	**Usuario**	Estudiante
Nombre historia	Inicio de sesión	**Responsable**	Jeffry Vera
Épica relacionada	E-01 Gestión de Usuarios	**Sprint asignado**	Sprint 1
Prioridad	Highest	**Riesgo**	Baja
Story points	5	**Estado**	Implementada
Descripción	Como estudiante, quiero iniciar sesión con mi cédula, para acceder al proceso de votación.		
Persistencia / módulo	Colecciones users y students; creación de SesionUsuario.		

Criterios de Aceptación

- Solicitar la cédula del estudiante.
- Verificar la existencia y estado activo del estudiante.
- Permitir acceso únicamente a personas registradas.
- Mostrar un mensaje específico cuando la cédula no exista.

Subtareas Técnicas

- Diseñar la ventana de autenticación.
- Programar la consulta de credenciales y creación de sesión.
- Probar accesos válidos, contraseña incorrecta y cédula inexistente.

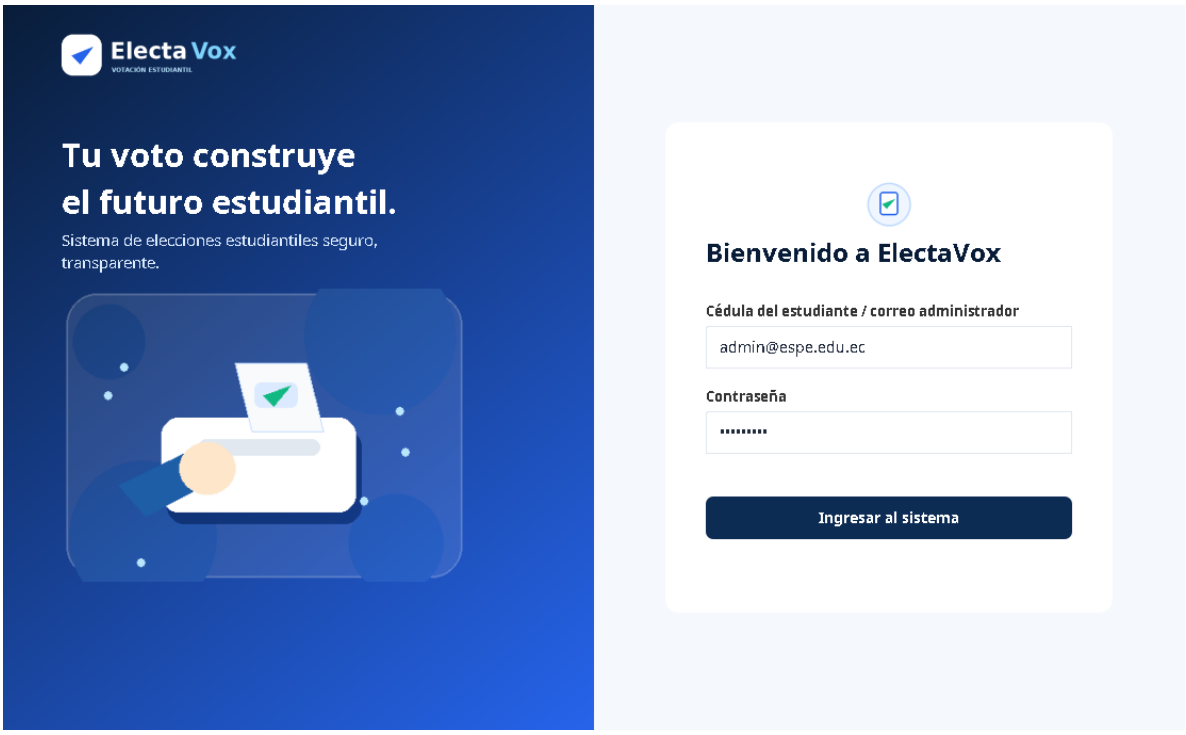
Desarrollo e Implementación de la Funcionalidad

FrmInicioSesion solicita cédula y contraseña, mientras ServicioAutenticacion consulta la colección users y contrasta el padrón activo. Al validar las credenciales se construye una SesionUsuario que determina el rol y habilita las opciones correspondientes en la ventana principal.

Como se observa en la Figura 8, la historia de usuario HU-02 fue implementada mediante la interfaz de inicio de sesión de ElectaVox. Esta pantalla permite ingresar al sistema utilizando la cédula del estudiante o el correo electrónico del administrador, junto con la contraseña correspondiente. Posteriormente, el sistema valida las credenciales y dirige al usuario al módulo adecuado según su rol, garantizando un acceso seguro y controlado.

Figura 8

Evidencia de HU-02 - Inicio de sesión



Validación del Resultado

La pantalla conserva la identidad visual de ElectaVox y separa claramente los campos de acceso. Ante datos no registrados o incorrectos se impide la apertura del panel principal y se comunica la causa al usuario.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Colecciones users y students; creación de SesionUsuario. La estructura de clases, formularios y servicios

permite identificar los componentes relacionados con HU-02 y reproducir las pruebas desde NetBeans.

Historia de Usuario 3: Registro de candidatos

Estructuración y Despliegue en Jira Software

La historia HU-03 fue vinculada con la épica E-02, asignada a Sprint 1 y estimada en 8 story points. Iker Guallo constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento. En la tabla 7 se muestra la historia de usuario con todos los detalles.

Tabla 7

Ficha de historia de usuario HU-03: Registro de candidatos

ID	HU-03	**Usuario**	Administrador
Nombre historia	Registro de candidatos	**Responsable**	Iker Guallo
Épica relacionada	E-02 Gestión de Candidatos	**Sprint asignado**	Sprint 1
Prioridad	Highest	**Riesgo**	Media
Story points	8	**Estado**	Implementada
Descripción	Como administrador, quiero registrar candidatos, para habilitar su participación en la elección.		
Persistencia / módulo	Colección candidates; fotografía almacenada como dato binario.		

Criterios de Aceptación

- Registrar nombre, carrera y propuestas.
- Permitir adjuntar una fotografía.
- Confirmar el almacenamiento correcto.
- Alertar cuando falte un campo obligatorio.

Subtareas Técnicas

- Diseñar el formulario y la tabla de candidatos.
- Programar el almacenamiento de datos y fotografía.
- Comprobar altas válidas y restricciones de campos.

Desarrollo e Implementación de la Funcionalidad

PanelCandidatos permite registrar número de lista, nombre, carrera, género, propuestas y fotografía. ServicioCandidatos transforma la imagen seleccionada a datos binarios y mantiene compatibilidad con registros anteriores que utilizaban el campo course.

Como se observa en la Figura 9, la historia de usuario HU-03 fue implementada mediante el módulo **Gestión de candidatos**, el cual permite al administrador registrar a los postulantes ingresando el número de lista, nombres completos, carrera, género, propuesta y fotografía. Además, la interfaz presenta una tabla de consulta y tarjetas informativas de los candidatos registrados, junto con opciones para actualizar o eliminar la información seleccionada.

Figura 9

Evidencia de HU-03 - Registro de candidatos



Validación del Resultado

La interfaz facilita la administración del catálogo electoral y muestra los candidatos registrados en una tabla. La presencia de número de lista y fotografía mejora la identificación visual durante la consulta y la votación.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Colección candidates; fotografía almacenada como dato binario. La estructura de clases, formularios y

servicios permite identificar los componentes relacionados con HU-03 y reproducir las pruebas desde NetBeans.

Historia de Usuario 4: Consulta de candidatos

Estructuración y Despliegue en Jira Software

La historia HU-04 fue vinculada con la épica E-02, asignada a Sprint 1 y estimada en 5 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento. A continuación, se muestra la Tabla 8, en la cual se puede apreciar con más detalle la estructura de la historia de usuario.

Tabla 8

Ficha de historia de usuario HU-04: Consulta de candidatos

ID	HU-04	**Usuario**	Estudiante
Nombre historia	Consulta de candidatos	**Responsable**	Jeffry Vera
Épica relacionada	E-02 Gestión de Candidatos	**Sprint asignado**	Sprint 1
Prioridad	Highest	**Riesgo**	Baja
Story points	5	**Estado**	Implementada
Descripción	Como estudiante, quiero visualizar candidatos y propuestas, para conocer las opciones antes de votar.		
Persistencia / módulo	Lectura de la colección candidates en la vista de candidatos y votación.		

Criterios de Aceptación

- Mostrar el listado completo de candidatos.
- Presentar fotografía y propuestas.
- Permitir recorrer las opciones registradas.

Subtareas Técnicas

- Diseñar las tarjetas o vista de consulta.
- Programar la carga de candidatos desde MongoDB.
- Probar la visualización de fotografías y textos extensos.

Desarrollo e Implementación de la Funcionalidad

La misma fuente documental que alimenta el módulo administrativo se reutiliza para la consulta estudiantil. Los registros se convierten en elementos visuales que exponen lista, nombre, carrera, fotografía y propuesta completa, evitando duplicar información entre pantallas.

Como se observa en la Figura 10, la historia de usuario HU-04 fue implementada mediante la sección **Consulta de candidatos**, donde se presentan los postulantes registrados con su número de lista, nombre, carrera y propuesta electoral. Esta información permite que el estudiante conozca las alternativas disponibles antes de emitir su voto. Además, la navegación centralizada en `FrmPrincipal` conserva el contexto de la sesión y garantiza la visualización de información actualizada.

Figura 10

Evidencia de HU-04 - Consulta de candidatos



Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Lectura de la colección candidates en la vista de candidatos y votación. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-04 y reproducir las pruebas desde NetBeans.

Historia de Usuario 5: Emitir voto

Estructuración y Despliegue en Jira Software

La historia HU-05 fue vinculada con la épica E-03, asignada a Sprint 1 y estimada en 13 story points. Iker Guallo constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 9

Ficha de historia de usuario HU-05: Emitir voto

ID	HU-05	**Usuario**	Estudiante
Nombre historia	Emitir voto	**Responsable**	Iker Guallo
Épica relacionada	E-03 Proceso de Votación	**Sprint asignado**	Sprint 1
Prioridad	Highest	**Riesgo**	Alta
Story points	13	**Estado**	Implementada
Descripción	Como estudiante, quiero emitir mi voto, para elegir a mi representante.		
Persistencia / módulo	Colección votes vinculada a electionId, studentId y candidateId.		

Criterios de Aceptación

- Permitir seleccionar un único candidato.
- Solicitar confirmación antes de guardar.
- Registrar el voto y actualizar el conteo general.

Subtareas Técnicas

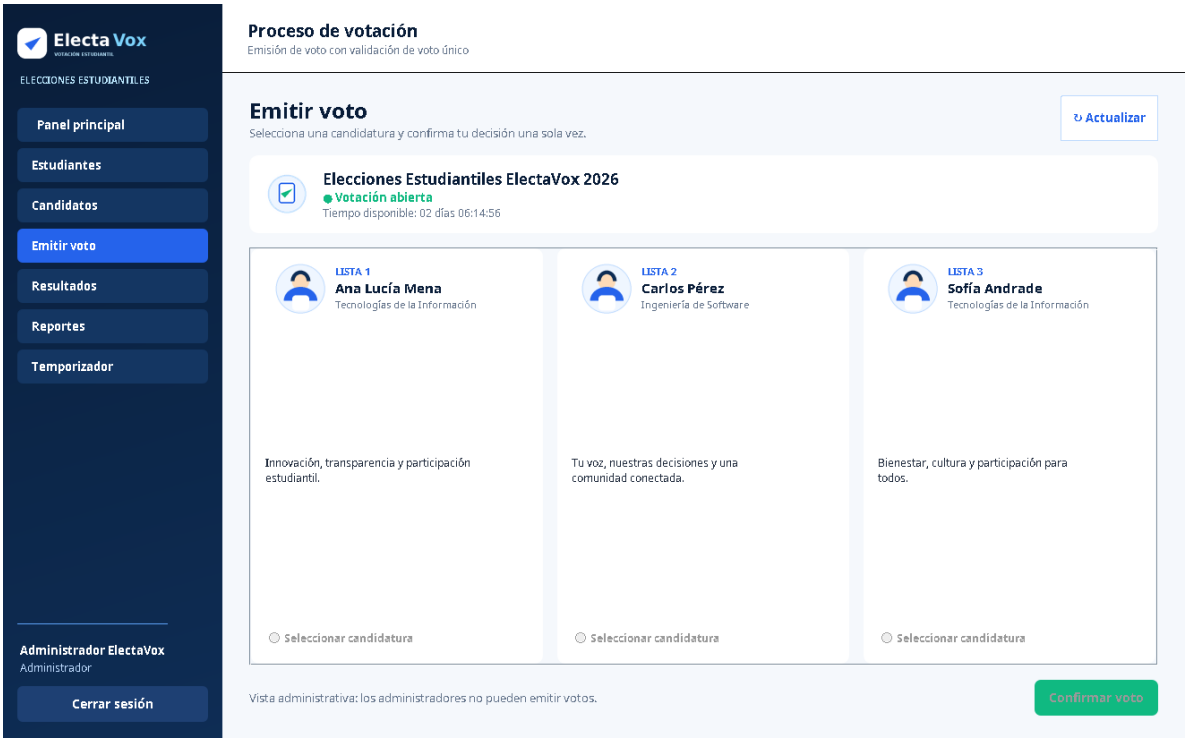
- Diseñar la interfaz de selección electoral.
- Programar la confirmación y persistencia del voto.
- Probar selección única, cancelación y registro exitoso.

Desarrollo e Implementación de la Funcionalidad

PanelVotacion carga los candidatos habilitados y controla que solo una opción permanezca seleccionada. Antes de persistir, ServicioVotacion solicita confirmación, valida la elección activa y registra la relación entre elección, estudiante y candidato con fecha y hora. En la Figura 11 La pantalla comunica de forma visible el candidato seleccionado y el tiempo restante. Una vez confirmado, el voto se almacena y los resultados pueden actualizarse sin intervención manual.

Figura 11

Evidencia de HU-05 - Emitir voto



Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Colección votes vinculada a electionId, studentId y candidateId. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-05 y reproducir las pruebas desde NetBeans.

Historia de Usuario 6: Validación de voto único

Estructuración y Despliegue en Jira Software

La historia HU-06 fue vinculada con la épica E-03, asignada a Sprint 1 y estimada en 8 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 10

Ficha de historia de usuario HU-06: Validación de voto único

ID	HU-06	**Usuario**	Sistema
Nombre historia	Validación de voto único	**Responsable**	Jeffry Vera
Épica relacionada	E-03 Proceso de Votación	**Sprint asignado**	Sprint 1
Prioridad	High	**Riesgo**	Media
Story points	8	**Estado**	Implementada
Descripción	Como sistema, quiero verificar si un estudiante ya votó, para evitar votos duplicados.		
Persistencia / módulo	Índice único compuesto en votes por electionId y studentId.		

Criterios de Aceptación

- Consultar el historial de votación del estudiante.
- Bloquear un segundo intento.
- Mostrar un mensaje informativo cuando ya exista un voto.

Subtareas Técnicas

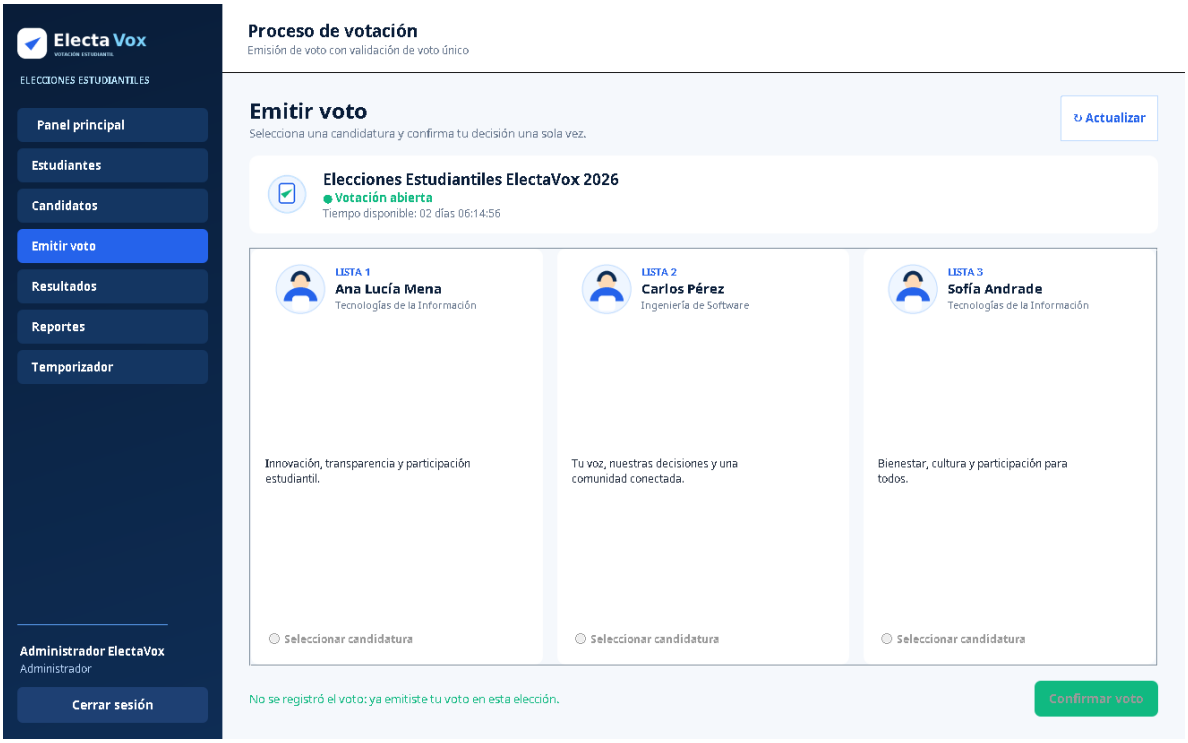
- Definir la restricción compuesta del voto.
- Programar la consulta previa y manejo de duplicados.
- Ejecutar una prueba de doble emisión con la misma sesión.

Desarrollo e Implementación de la Funcionalidad

La protección se aplica en dos niveles: una consulta previa informa al usuario y una restricción única en MongoDB impide que una condición de concurrencia genere un segundo documento. El servicio captura el conflicto y mantiene intacto el conteo electoral.

Figura 12

Evidencia de HU-06 - Validación de voto único



Validación del Resultado

La evidencia muestra el estado de votación ya registrada y los controles inhabilitados. De este modo, la transparencia no depende únicamente de la interfaz, sino también de una regla persistente en la base de datos.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Índice único compuesto en votes por electionId y studentId. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-06 y reproducir las pruebas desde NetBeans.

Historia de Usuario 7: Temporizador electoral

Estructuración y Despliegue en Jira Software

La historia HU-07 fue vinculada con la épica E-03, asignada a Sprint 1 y estimada en 5 story points. Iker Guallo constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 11

Ficha de historia de usuario HU-07: Temporizador electoral

ID	HU-07	**Usuario**	Estudiante
Nombre historia	Temporizador electoral	**Responsable**	Iker Guallo
Épica relacionada	E-03 Proceso de Votación	**Sprint asignado**	Sprint 1
Prioridad	High	**Riesgo**	Baja
Story points	5	**Estado**	Implementada
Descripción	Como estudiante, quiero observar un contador regresivo, para conocer el tiempo restante de la votación.		
Persistencia / módulo	Fechas de inicio y fin en elections; actualización Swing Timer.		

Criterios de Aceptación

- Mostrar el tiempo restante.
- Actualizar el contador cada segundo.
- Cerrar automáticamente la votación al llegar a cero.

Subtareas Técnicas

- Diseñar el indicador visual del tiempo.
- Programar la actualización periódica y el cálculo restante.
- Probar cambio de segundos y cierre automático.

Desarrollo e Implementación de la Funcionalidad

La elección conserva sus fechas de inicio y fin en la colección elections. La interfaz calcula la diferencia con la hora actual y un temporizador de Swing refresca el indicador cada segundo. Al alcanzar cero se ejecuta el cierre y se desactivan inmediatamente los controles de votación.

Figura 13

Evidencia de HU-07 - Temporizador electoral



Validación del Resultado

El contador es dinámico y permite observar el transcurso real del tiempo. La sincronización entre estado persistente y componentes visuales evita que una pantalla abierta continúe aceptando votos fuera del período.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Fechas de inicio y fin en elections; actualización Swing Timer. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-07 y reproducir las pruebas desde NetBeans.

Historia de Usuario 8: Resultados en tiempo real

Estructuración y Despliegue en Jira Software

La historia HU-08 fue vinculada con la épica E-04, asignada a Sprint 2 y estimada en 13 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 12

Ficha de historia de usuario HU-08: Resultados en tiempo real

ID	HU-08	**Usuario**	Administrador
Nombre historia	Resultados en tiempo real	**Responsable**	Jeffry Vera
Épica relacionada	E-04 Resultados y Estadísticas	**Sprint asignado**	Sprint 2
Prioridad	Medium	**Riesgo**	Media
Story points	13	**Estado**	Implementada
Descripción	Como administrador, quiero visualizar resultados en tiempo real, para monitorear el avance de la elección.		
Persistencia / módulo	Agregaciones sobre votes y candidates; refresco cada dos segundos.		

Criterios de Aceptación

- Mostrar barras con los resultados.
- Actualizar automáticamente la información.
- Presentar total y porcentaje por candidato.

Subtareas Técnicas

- Diseñar el tablero de resultados.
- Programar agregaciones y refresco automático.
- Verificar totales, porcentajes y actualización periódica.

Desarrollo e Implementación de la Funcionalidad

ServicioResultados agrupa los votos por candidato y calcula porcentajes respecto del total emitido. PanelResultados representa cada valor mediante barras de progreso y actualiza la información automáticamente cada dos segundos mientras la ventana permanece activa.

Figura 14

Evidencia de HU-08 - Resultados en tiempo real



Validación del Resultado

El administrador puede observar listas, votos y porcentajes sin generar consultas manuales. Los datos demostrativos de la captura permiten validar la proporcionalidad de las barras y la coherencia entre conteo y porcentaje.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Agregaciones sobre votes y candidates; refresco cada dos segundos. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-08 y reproducir las pruebas desde NetBeans.

Historia de Usuario 9: Estadísticas por curso

Estructuración y Despliegue en Jira Software

La historia HU-09 fue vinculada con la épica E-04, asignada a Sprint 2 y estimada en 8 story points. Iker Guallo constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 13

Ficha de historia de usuario HU-09: Estadísticas por curso

ID	HU-09	**Usuario**	Administrador
Nombre historia	Estadísticas por curso	**Responsable**	Iker Guallo
Épica relacionada	E-04 Resultados y Estadísticas	**Sprint asignado**	Sprint 2
Prioridad	Medium	**Riesgo**	Media
Story points	8	**Estado**	Implementada
Descripción	Como administrador, quiero visualizar estadísticas por curso, para analizar la participación estudiantil.		
Persistencia / módulo	Cruce de students habilitados con votes mediante la cédula del estudiante.		

Criterios de Aceptación

- Mostrar la cantidad de votantes por curso.
- Calcular el porcentaje de participación.
- Presentar la información de forma ordenada.

Subtareas Técnicas

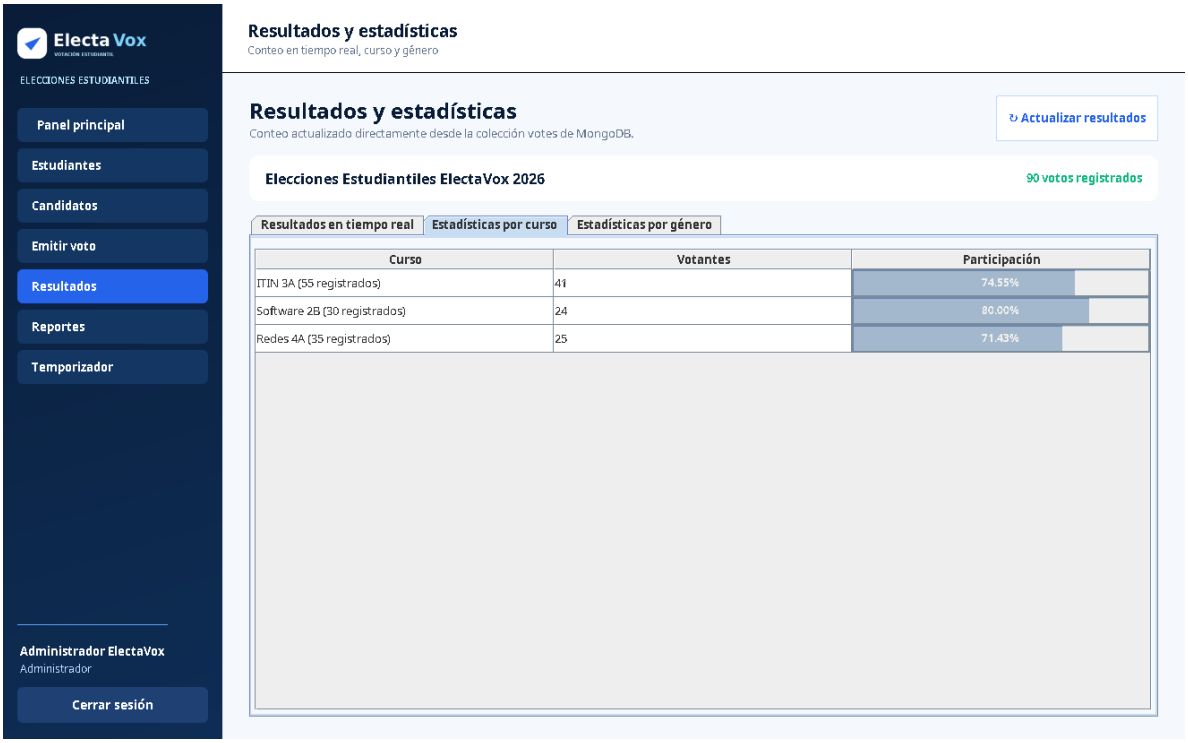
- Diseñar la tabla de participación por curso.
- Programar el cruce entre padrón y votos.
- Probar cursos con y sin participación.

Desarrollo e Implementación de la Funcionalidad

El cálculo toma como denominador el total de estudiantes habilitados en cada curso y como numerador los estudiantes con voto registrado en la elección. La tabla incluye valores absolutos y una barra porcentual para facilitar la comparación entre grupos.

Figura 15

Evidencia de HU-09 - Estadísticas por curso



Validación del Resultado

La vista permite identificar cursos con mayor o menor participación. Al considerar el padrón habilitado, el porcentaje representa una tasa real y no solamente la distribución de los votos emitidos.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Cruce de students habilitados con votes mediante la cédula del estudiante. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-09 y reproducir las pruebas desde NetBeans.

Historia de Usuario 10: Estadísticas por género

Estructuración y Despliegue en Jira Software

La historia HU-10 fue vinculada con la épica E-04, asignada a Sprint 2 y estimada en 8 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 14

Ficha de historia de usuario HU-10: Estadísticas por género

ID	HU-10	**Usuario**	Administrador
Nombre historia	Estadísticas por género	**Responsable**	Jeffry Vera
Épica relacionada	E-04 Resultados y Estadísticas	**Sprint asignado**	Sprint 2
Prioridad	Medium	**Riesgo**	Media
Story points	8	**Estado**	Implementada
Descripción	Como administrador, quiero visualizar estadísticas por género, para analizar la participación estudiantil.		
Persistencia / módulo	Agrupación de students y votes por el campo genero.		

Criterios de Aceptación

- Mostrar la participación por género.
- Calcular porcentajes automáticamente.
- Usar un componente gráfico.

Subtareas Técnicas

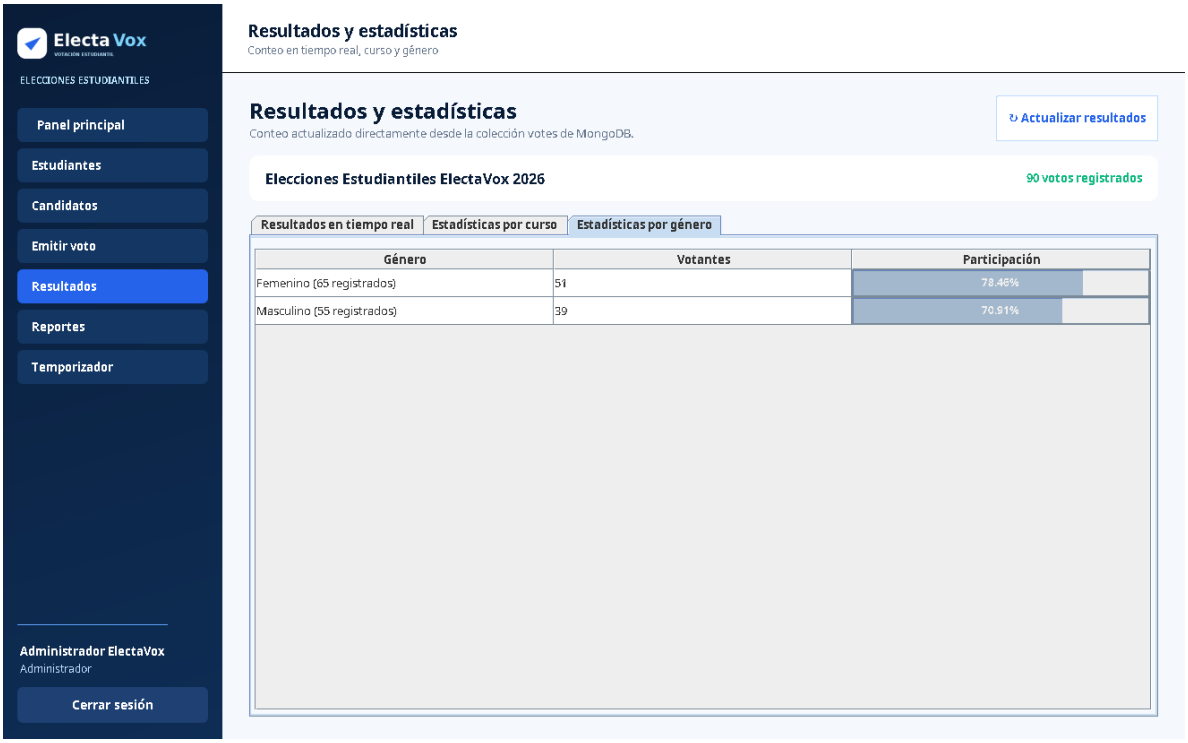
- Diseñar la tabla gráfica por género.
- Programar la agrupación de estudiantes y votos.
- Verificar porcentajes y categoría no especificada.

Desarrollo e Implementación de la Funcionalidad

El servicio normaliza el valor de género y contempla la categoría No especifica para registros incompletos. Después calcula votantes, habilitados y porcentaje para cada grupo; la interfaz representa el resultado mediante barras dentro de la tabla.

Figura 16

Evidencia de HU-10 - Estadísticas por género



Validación del Resultado

La estadística ofrece una lectura demográfica del proceso sin alterar la confidencialidad del voto, pues únicamente relaciona participación y padrón, no la elección individual de cada estudiante.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Agrupación de students y votes por el campo genero. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-10 y reproducir las pruebas desde NetBeans.

Historia de Usuario 11: Generar reporte final

Estructuración y Despliegue en Jira Software

La historia HU-11 fue vinculada con la épica E-05, asignada a Sprint 2 y estimada en 5 story points. Iker Guallo constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 15

Ficha de historia de usuario HU-11: Generar reporte final

ID	HU-11	**Usuario**	Administrador
Nombre historia	Generar reporte final	**Responsable**	Iker Guallo
Épica relacionada	E-05 Reportes y Cierre Electoral	**Sprint asignado**	Sprint 2
Prioridad	Low	**Riesgo**	Baja
Story points	5	**Estado**	Implementada
Descripción	Como administrador, quiero generar un reporte final, para conservar evidencia de la elección.		
Persistencia / módulo	Lectura consolidada de elections, votes, candidates y students; salida TXT.		

Criterios de Aceptación

- Generar un archivo con resultados finales.
- Incluir estadísticas generales.
- Registrar fecha y hora del cierre.

Subtareas Técnicas

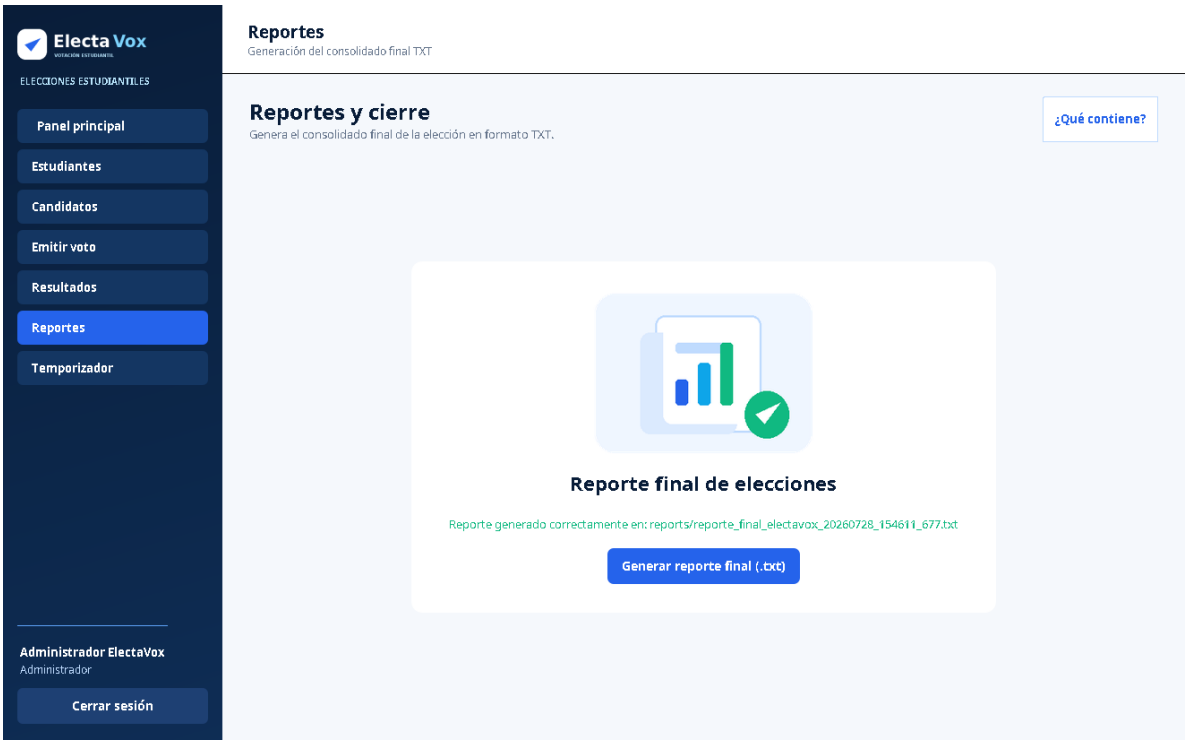
- Diseñar la vista de reportes.
- Programar la construcción y escritura del archivo TXT.
- Validar contenido, nombre y ruta del archivo generado.

Desarrollo e Implementación de la Funcionalidad

ServicioReportes construye un documento de texto con período electoral, fecha de generación, estado, participación general, resultados por candidato y estadísticas por curso y género. La generación puede ejecutarse manualmente desde PanelReportes o automáticamente al cerrar la elección.

Figura 17

Evidencia de HU-11 - Generar reporte final



Validación del Resultado

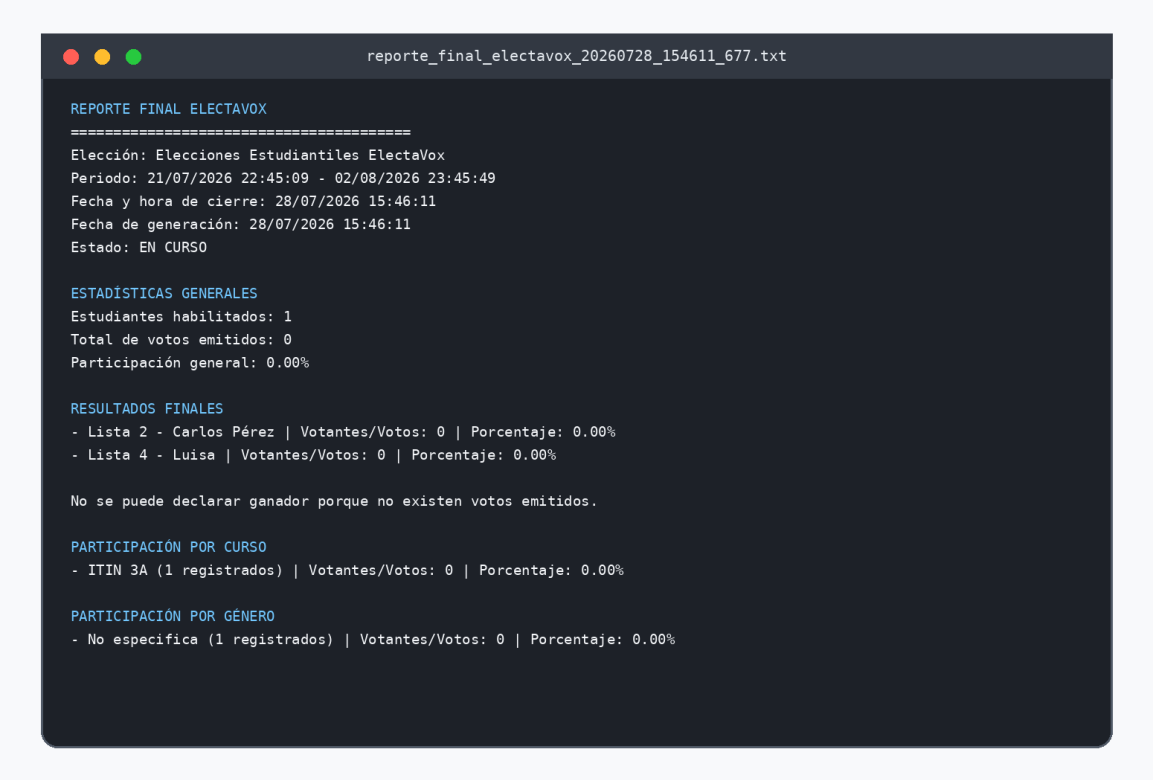
La interfaz confirma la ruta del archivo y ofrece una vista previa del contenido. El documento generado constituye un respaldo legible, portable y verificable para la entrega del acta electoral.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Lectura consolidada de elections, votes, candidates y students; salida TXT. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-11 y reproducir las pruebas desde NetBeans.

Figura 18

Contenido del Reporte Final Generado por ElectaVox



Historia de Usuario 12: Cierre de elección

Estructuración y Despliegue en Jira Software

La historia HU-12 fue vinculada con la épica E-05, asignada a Sprint 2 y estimada en 8 story points. Jeffry Vera constó como responsable principal de su ejecución. En Jira se mantuvieron las subtareas de diseño, programación y pruebas para facilitar el seguimiento del incremento.

Tabla 16

Ficha de historia de usuario HU-12: Cierre de elección

ID	HU-12	**Usuario**	Administrador
Nombre historia	Cierre de elección	**Responsable**	Jeffry Vera
Épica relacionada	E-05 Reportes y Cierre Electoral	**Sprint asignado**	Sprint 2
Prioridad	Low	**Riesgo**	Media
Story points	8	**Estado**	Implementada
Descripción	Como administrador, quiero cerrar oficialmente la elección, para impedir nuevos votos y declarar al ganador.		
Persistencia / módulo	Actualización de elections y generación de acta TXT.		

Criterios de Aceptación

- Bloquear nuevos votos al cerrar.
- Identificar automáticamente al candidato ganador.
- Generar un acta de cierre con resultados.

Subtareas Técnicas

- Diseñar los controles de cierre electoral.
- Programar cambio de estado, ganador y generación del acta.
- Probar cierre manual, automático y empate.

Desarrollo e Implementación de la Funcionalidad

ServicioEleccion cambia el estado de la elección y ServicioVotacion consulta dicho estado antes de cualquier inserción. El cierre consolida resultados, identifica al ganador o declara empate y solicita a ServicioReportes la generación del acta correspondiente.

Figura 19

Evidencia de HU-12 - Cierre de elección



Validación del Resultado

La pantalla de administración permite cerrar el proceso de forma controlada. El mismo flujo se invoca automáticamente cuando el temporizador llega a cero, garantizando consistencia entre el límite temporal y la recepción de votos.

Respaldo y Trazabilidad Técnica

La funcionalidad se encuentra integrada en el proyecto Maven y se apoya en Actualización de elections y generación de acta TXT. La estructura de clases, formularios y servicios permite identificar los componentes relacionados con HU-12 y reproducir las pruebas desde NetBeans.

Módulo de Seguridad y Acceso (Login)

Aunque HU-02 describe el ingreso del estudiante, la seguridad de ElectaVox opera de forma transversal. Las credenciales se mantienen separadas del padrón; la sesión conserva identidad, nombre y rol; y FrmPrincipal adapta la navegación según los permisos. Los servicios verifican el estado del usuario antes de permitir operaciones sensibles, por lo que ocultar un botón no constituye la única medida de control.

Flujo de Autenticación

- Ingreso de cédula y contraseña en FrmInicioSesion.
- Consulta del usuario y verificación de la clave mediante ServicioAutenticacion.
- Comprobación de que el estudiante se encuentre activo en el padrón.
- Creación de SesionUsuario y apertura de FrmPrincipal.
- Configuración del menú para administrador o estudiante.

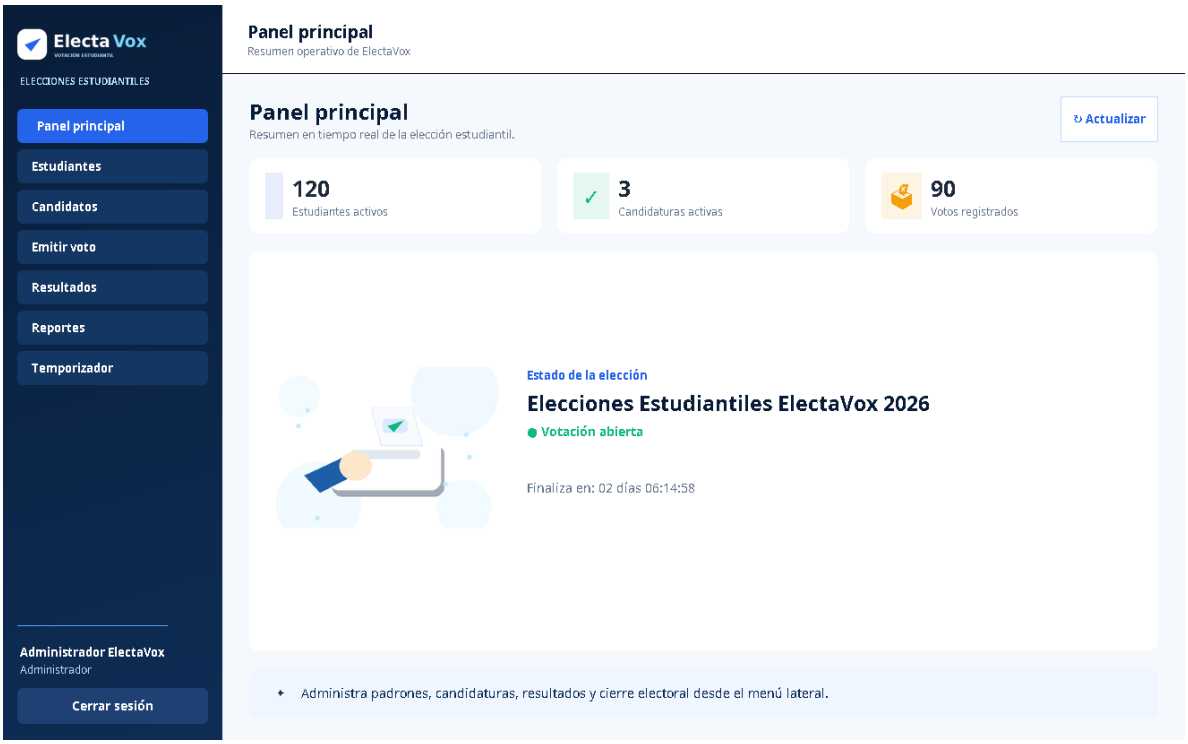
Las validaciones de formato se realizan antes de consultar la base de datos y los mensajes evitan revelar información innecesaria. De este modo, el sistema combina retroalimentación suficiente para el usuario con una separación clara de responsabilidades.

Módulo de Interfaz Principal y Contenedor de Navegación

FrmPrincipal funciona como contenedor de los paneles del sistema. El menú lateral presenta Inicio, Estudiantes, Candidatos, Votar, Resultados y Reportes, mientras la zona central se reemplaza dinámicamente sin abrir múltiples ventanas. El encabezado identifica al usuario y su rol, y el reloj visible actualiza la hora de forma continua.

Figura 20

Interfaz Principal y Contenedor de Navegación de ElectaVox



Los paneles implementan una interfaz común de actualización para recargar datos cuando el usuario cambia de módulo. Esta decisión evita mostrar información obsoleta y centraliza la experiencia visual mediante TemaInterfaz, PanelRedondeado, PanelDegradado, BotonRedondeado y TarjetaMetrica.

Persistencia de Datos y Estructura de MongoDB

La configuración de MongoDB se concentra en ConfiguracionMongo e InicializadorBaseDatos. Al iniciar, el sistema obtiene la cadena de conexión y el nombre de la base desde application.properties, abre el cliente y asegura la existencia de índices requeridos. Los servicios reciben acceso a las colecciones y transforman los documentos en estructuras utilizadas por la interfaz.

Tabla 17

Colecciones Principales de la Base de Datos ElectaVox

Colección	Propósito	Campos principales	Control de integridad
users	Autenticación y roles	cedula, passwordHash, role, active	Cédula única
students	Padrón electoral	cedula, nombreCompleto, curso, genero, correo	Cédula y correo únicos
candidates	Opciones electorales	numeroLista, nombre, carrera, propuestas, foto	Número de lista validado
elections	Período y estado	nombre, fechaInicio, fechaFin, estado	Una elección activa controlada
votes	Votos emitidos	electionId, studentId, candidateId, fechaHora	Índice único elección-estudiante

La persistencia no almacena únicamente los resultados; también respalda el estado temporal de la elección, las credenciales y la información necesaria para calcular participación. El reporte final se genera a partir de consultas consolidadas, sin depender de valores escritos manualmente en la interfaz.

Pruebas Funcionales Realizadas

Cada historia fue contrastada con sus criterios de aceptación mediante pruebas funcionales sobre el sistema. Se utilizaron datos demostrativos para comprobar validaciones, cambios de estado, agregaciones y salida documental. La Tabla siguiente resume los casos de mayor riesgo y el resultado obtenido.

Tabla 18

Casos de Prueba Ejecutados sobre ElectaVox

Caso de prueba	Resultado esperado	Resultado obtenido	Estado
Registro con cédula repetida	Bloquear el alta y mostrar alerta.	El servicio detectó la cédula existente y no insertó un nuevo documento.	Conforme
Inicio con cédula no registrada	Informar que la cédula no existe.	La autenticación fue rechazada con mensaje específico.	Conforme
Emisión de un segundo voto	Bloquear cualquier voto duplicado.	La consulta previa y el índice compuesto impidieron la inserción.	Conforme
Actualización del temporizador	Cambiar cada segundo y cerrar en cero.	El contador fue dinámico y deshabilitó la votación al finalizar.	Conforme

Caso de prueba	Resultado esperado	Resultado obtenido	Estado
Resultados y estadísticas	Calcular conteos y porcentajes correctos.	Las tablas y barras reflejaron el padrón y los votos demostrativos.	Conforme
Generación de reporte	Crear TXT con resultados, estadísticas y fecha.	Se creó el archivo y se mostró la ruta y vista previa.	Conforme
Cierre electoral	Bloquear votos, obtener ganador o empate y generar acta.	El estado fue actualizado y se produjo la documentación de cierre.	Conforme

Matriz de Implementación y Resultados

La matriz consolida el cumplimiento del Product Backlog y facilita la trazabilidad entre la necesidad, el componente construido y la evidencia presentada.

Tabla 19

Matriz de Cumplimiento de Historias de Usuario

HU	Funcionalidad	Implementación verificada	Resultado
HU-01	Registro de estudiantes	PanelEstudiantes, ServicioEstudiantes, students/users	Cumplida
HU-02	Inicio de sesión	FrmInicioSesion, ServicioAutenticacion, SesionUsuario	Cumplida
HU-03	Registro de candidatos	PanelCandidatos, ServicioCandidatos, candidates	Cumplida
HU-04	Consulta de candidatos	Carga de tarjetas y datos desde candidates	Cumplida
HU-05	Emitir voto	PanelVotacion y ServicioVotacion	Cumplida
HU-06	Voto único	Consulta previa e índice compuesto en votes	Cumplida
HU-07	Temporizador	Swing Timer y fechas de elections	Cumplida
HU-08	Resultados en tiempo real	ServicioResultados y barras porcentuales	Cumplida
HU-09	Estadísticas por curso	Cruce de padrón y votos por curso	Cumplida
HU-10	Estadísticas por género	Agrupación y porcentajes por género	Cumplida
HU-11	Reporte final	ServicioReportes y archivo TXT	Cumplida
HU-12	Cierre de elección	ServicioEleccion, bloqueo y acta	Cumplida

El conjunto de evidencias muestra que el incremento final cubre las cinco épicas. Además de implementar la funcionalidad visible, el proyecto incorpora reglas persistentes, actualización automática y salida documental, aspectos necesarios para que el sistema pueda demostrarse de principio a fin.

Conclusiones

La construcción de ElectaVox permitió transformar las doce historias de usuario del Product Backlog en módulos funcionales integrados, manteniendo correspondencia entre épicas, sprints, responsables, criterios de aceptación y evidencias del sistema.

La combinación de Java Swing, NetBeans y Maven proporcionó una base adecuada para desarrollar interfaces editables y organizar los componentes técnicos, mientras que la separación en servicios redujo el acoplamiento entre la presentación y la persistencia.

MongoDB permitió conservar el padrón, los candidatos, las elecciones y los votos mediante documentos e índices. La restricción única compuesta de la colección votes fortaleció la garantía de voto único más allá de las validaciones visuales.

El temporizador, la actualización automática de resultados y el cierre electoral consolidaron un flujo coherente: la elección se habilita durante un período definido, recibe votos válidos, bloquea duplicidades, calcula participación y produce evidencia al finalizar.

Jira facilitó la organización del trabajo en dos sprints y la descomposición de cada historia en diseño, programación y pruebas, permitiendo visualizar la trazabilidad desde la planificación hasta el incremento entregado.

Recomendaciones

- Mantener una copia de respaldo de la base de datos y del proyecto antes de la defensa, y verificar la cadena de conexión configurada en application.properties.
- Ejecutar una prueba integral con varios estudiantes, cursos, géneros y candidatos para demostrar con mayor variedad los gráficos y porcentajes.
- Conservar los índices únicos de cédula, correo y voto por elección-estudiante al migrar la base de datos a otro equipo o entorno.
- Registrar en Jira cualquier incidencia o mejora posterior, vinculándola con la épica correspondiente para mantener la trazabilidad del producto.
- Considerar en una futura iteración la exportación adicional a PDF, auditoría de eventos y mecanismos de respaldo automático de MongoDB.

Referencias

Apache Software Foundation. (s.f.). Designing a Swing GUI in NetBeans IDE. Apache NetBeans. https://netbeans.apache.org/

Atlassian. (2020). Jira Software: backlog, sprints e historias de usuario. https://www.atlassian.com/software/jira

GitHub, Inc. (s.f.). GitHub Docs: repositorios, ramas y commits. https://docs.github.com/

MongoDB, Inc. (2024). MongoDB Java Sync Driver documentation. https://www.mongodb.com/docs/drivers/java/sync/current/

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum. Scrum.org. https://scrumguides.org/

Enlaces del Proyecto

Tablero del proyecto en Jira Software:

https://grupo-10-proyecto.atlassian.net/jira/software/projects/SCRUM/summary